

ChessCoachAgent

Deep-Dive Technical Guide

LangGraph · LangChain · Stockfish · FastAPI · WebSockets

This guide is written so you can read it, understand every design decision, and then rebuild the entire project from a blank directory with minimal outside help. It covers the libraries, the architecture, the code, and the reasoning behind each choice.

PART 1 — THE LIBRARIES

LangChain — What It Actually Does

LangChain is not one thing — it is a collection of small building blocks for working with language models. The piece this project uses most is **ChatAnthropic**, which wraps Anthropic's Claude API and gives you a consistent Python interface.

Before LangChain existed you would call the Anthropic SDK directly. LangChain adds value because it defines a universal message format and a consistent streaming interface that works the same way regardless of which AI company's model you are using.

ChatAnthropic

This is what the Analyst and Coach agents use to talk to Claude. You instantiate it once with a model name and temperature, then call **.stream(prompt)** to get tokens one at a time as Claude generates them.

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-6", temperature=0.3)

# .stream() yields chunk objects; chunk.content is the text token
full = ""
for chunk in llm.stream("Explain the Sicilian Defence in 2 sentences."):
    full += chunk.content
    print(chunk.content, end="", flush=True)
```

Temperature controls how creative vs. deterministic the output is. 0.1 = very consistent, predictable (used by the Critic for scoring). 0.5 = balanced (Coach). 0.3 = slightly creative (Analyst). You do not need higher temperatures for factual, structured tasks.

The Messages Format

Under the hood, every LangChain call sends a list of messages. A message has a **role** (system / human / assistant) and **content** (text). In this project the agents send a single human message containing the full formatted prompt — no multi-turn conversation history. The LangGraph state has a **messages** field managed by LangGraph itself, but the individual agents do not use it.

Why not use multi-turn history? Each agent in this pipeline does a single large analysis task, not a back-and-forth chat. Sending the entire game data plus all prior conversation would waste tokens and confuse the model.

LangGraph — Orchestrating AI Workflows

LangGraph is a separate library built on top of LangChain that lets you define an AI workflow as a **directed graph**. Each node in the graph is a Python function. Arrows (edges) say which node runs next. The graph can branch and loop — this is what makes it more powerful than a simple sequential pipeline.

The Core Concept: StateGraph + TypedDict

Every LangGraph application has a **state object** — a Python TypedDict that holds all the data that flows through the pipeline. Every node receives the current state, does its work, and returns a dict containing only the keys it wants to update. LangGraph merges those updates back into the shared state before passing it to the next node.

```
from typing import TypedDict
from langgraph.graph import StateGraph, END, START

class MyState(TypedDict):
    input_text: str
    result: str
    attempts: int

def step_one(state: MyState) -> dict:
    # Return only what this node changes
    return {"result": state["input_text"].upper()}

def step_two(state: MyState) -> dict:
    return {"attempts": state["attempts"] + 1}

workflow = StateGraph(MyState)
workflow.add_node("step_one", step_one)
workflow.add_node("step_two", step_two)
workflow.add_edge(START, "step_one")
workflow.add_edge("step_one", "step_two")
workflow.add_edge("step_two", END)

graph = workflow.compile()
result = graph.invoke({"input_text": "hello", "result": "", "attempts": 0})
# result["result"] == "HELLO", result["attempts"] == 1
```

Conditional Edges — Branching

A conditional edge calls a **router function** that inspects the current state and returns a string. That string maps to the next node. This is how the Coach-Critic feedback loop is implemented: after the Critic runs, a router checks whether the report was approved and either loops back to the Coach or proceeds to the final step.

```
def should_regenerate(state: MyState) -> str:
    # Return a string key that maps to a node name
    if state["critique_result"]["approved"]:
        return "approved"
    return "regenerate"

workflow.add_conditional_edges(
    "critic_review",          # source node
    should_regenerate,       # router function
    {
        "regenerate": "generate_lesson",  # if "regenerate" → go back to coach
        "approved":   "format_output",    # if "approved" → move forward
    }
)
```

compile() and invoke()

compile() validates the graph structure (every node must be reachable, every edge must point to a real node) and builds an internal execution plan. It returns a runnable object. Call **invoke(initial_state)** to run the graph synchronously from START to END. The graph runs in a single thread — each node blocks until complete before the next begins.

***Important:** In this project the entire LangGraph graph runs in a background thread (via ThreadPoolExecutor) so the FastAPI async event loop stays free. The graph is fully synchronous — it has no async nodes.*

PART 2 — THE PROJECT ARCHITECTURE

The Full Graph Topology

There are two compiled graphs: one for the CLI and one for the web. They share the same six core nodes. The CLI adds two more at the front for the terminal username prompt and game-count questions.

CLI graph:

```
START
→ memory_check      (terminal: asks for Chess.com username)
→ analyst_interview (terminal: asks how many games, what time control)
→ fetch_games       ■■
→ analyze_games      ■
→ coach_interview    ■ ← shared core
→ generate_lesson    ■
→ critic_review      ■■■■■■
    ■                ■
    ■■■ "regenerate" → generate_lesson (loop back, up to 3x)
    ■■■ "approved"  → format_output
→ END
```

Web graph:

```
START
→ fetch_games (username/num_games/time_control already in state from the form)
→ (same core from here...)
→ END
```

Why Two Graphs?

The CLI version needs to ask the user questions via the terminal. The web version receives all that information up front from the HTML form, so those interactive nodes are skipped entirely. Rather than adding if/else branches inside nodes, the cleaner approach is to compile two separate graphs that share the same underlying node functions.

The Shared State: ChessCoachState

Every node reads from and writes to this single TypedDict. Here is every field and why it exists:

```

class ChessCoachState(TypedDict):
    # Who the user is
    username: str
    is_returning_user: bool
    user_data: dict          # SQLite row (empty dict for new users)

    # What the user asked for (set by form or CLI interview)
    num_games: int
    time_control: str        # blitz | bullet | rapid | daily | all

    # Raw and parsed game data
    raw_games: list          # raw dicts from Chess.com API
    game_summaries: list     # dicts from pgn_parser.parse_game()

    # What each agent produced
    analysis_report: str     # Analyst's markdown output
    player_goals: dict       # Coach's interview results
    coaching_report: str     # Coach's plan (may be rewritten if Critic rejects)
    critique_result: dict    # Critic's score: {approved, total_score, feedback, ...}

    # Control flow
    critique_attempts: int   # how many times Coach has run (max 3)
    final_report: str        # assembled markdown + PDF path

    # LangGraph-managed message history
    messages: Annotated[list[BaseMessage], add_messages]

```

The **messages** field uses a special LangGraph annotation: **add_messages**. Normally when a node returns a key, LangGraph replaces the old value. With **Annotated[list, add_messages]** it appends instead of replacing. This is designed for chat history — each node can add messages without overwriting what previous nodes added. In this project no node actually writes to messages directly; it is there for future use.

The EventBus Pattern

This is the most important design decision in the project. All three agents need to send output to the user — but in CLI mode that means **print()**, and in web mode it means **WebSocket.send_json()**. If agents had hardcoded **print()** calls, the web version would break. If they had hardcoded **WebSocket** calls, the CLI version would break.

The EventBus is a **singleton object** (one instance shared by the whole process) that acts as the single output channel. Every agent calls **bus.debug()**, **bus.stream_chunk()**, **bus.ask()**, etc. The bus decides how to handle each call based on whether it is in 'cli' or 'web' mode.

```

# utils/events.py (simplified)
class EventBus:
    def __init__(self):
        self._mode = "cli"
        self._event_q = None    # Queue: graph → WebSocket
        self._input_q = None    # Queue: WebSocket → graph

    def setup_web(self, event_q, input_q):
        self._mode = "web"
        self._event_q = event_q
        self._input_q = input_q

    def debug(self, message: str):
        if self._mode == "web":
            self._event_q.put({"type": "debug", "message": message})
        else:
            print(f" {message}")

    def ask(self, question_id: str, question: str, options=None) -> str:
        self.show_question(question_id, question, options)
        if self._mode == "web":
            return self._input_q.get()    # BLOCKS until user submits answer
        else:
            return input(" > ").strip()

bus = EventBus()    # module-level singleton

```

How the Web Bridge Works

When the web server receives a WebSocket connection it creates two **queue.Queue** objects and calls **bus.setup_web(event_q, input_q)**. Then it launches the LangGraph graph in a background thread. The main async loop polls event_q and forwards messages to the browser. When the Coach asks a question, **bus.ask()** blocks the background thread on input_q.get(). The browser shows the question, the user submits an answer, the WebSocket handler puts it on input_q, and the blocked thread resumes.

```
# api/server.py (simplified bridge loop)
event_q = queue.Queue()
input_q = queue.Queue()
bus.setup_web(event_q, input_q)

future = loop.run_in_executor(executor, run_session, start_data) # background thread

while True:
    while not event_q.empty():
        await websocket.send_json(event_q.get_nowait()) # forward to browser

    if future.done() and event_q.empty():
        break

    try:
        msg = await asyncio.wait_for(websocket.receive_json(), timeout=0.05)
        if msg.get("type") == "answer":
            input_q.put(msg["value"]) # unblocks bus.ask() in background thread
    except asyncio.TimeoutError:
        pass

    await asyncio.sleep(0.02)
```

Thread safety: The `EventBus` uses a `threading.Lock` to protect reads and writes to the queue references. This matters because the `setup_web()` call happens on the `async` thread while the graph runs on a `ThreadPoolExecutor` thread.

PART 3 — THE THREE AGENTS

Agent 1: The Analyst

File: `agents/analyst.py` — function: `analyze_games_node(state)`

The Analyst's job is to read all the parsed game data and find *patterns across games*, not just comment on a single game. It does not use LangGraph features beyond being a node — it is a single Claude API call with a carefully structured prompt.

What It Does

- Calls `bus.node_active('analyst')` so the web UI lights up the Analyst box.
- Formats all `game_summaries` into a readable text block with opening names, result, move sequences, and ■ time-pressure markers.
- Sends one large prompt to Claude via `llm.stream()`, accumulating the response token by token.
- Each token is forwarded to the browser with `bus.stream_chunk('analyst', text)` — this is the live typing effect.
- Stores the complete response in `analysis_report` for the Coach to read.

The Prompt Structure

The prompt tells Claude to produce exactly five sections: Opening Patterns, Middlegame Patterns, Endgame Patterns, Time Management, and Top 3 Priority Areas. The instruction *'Be specific — reference move numbers or game patterns, not generic advice that could apply to anyone'* is critical. Without this constraint, language models tend to give boilerplate advice.

Time Pressure Detection

Chess.com PGNs embed clock times in move comments: [%clk 0:03:22]. The PGN parser extracts this with a regex and flags any move where the player had less than 30 seconds remaining. These moves get a ■ marker in the move string sent to Claude. The Analyst and Coach are both instructed to *deprioritize* errors made under time pressure — those are training issues, not pattern issues.

```
# tools/pgn_parser.py
def _extract_clock(comment: str) -> Optional[int]:
    match = re.search(r'\[%clk (\d+):(\d+):(\d+)\]', comment)
    if match:
        h, m, s = int(match.group(1)), int(match.group(2)), int(match.group(3))
        return h * 3600 + m * 60 + s
    return None
```

Agent 2: The Coach

File: **agents/coach.py** — two functions: **coach_interview_node** and **generate_lesson_node**

The Coach is split into two LangGraph nodes because LangGraph needs a clean separation between 'gather input' and 'produce output'. Combining them would mean the node runs for a long time with a blocking user interaction in the middle — harder to reason about and harder to retry.

coach_interview_node — Collecting Goals

This node calls `bus.ask()` four times, once for each question: main goal, white opening preference, Black vs e4 response, and Black vs d4 response. Each answer is stored in the goals dict and written back to state as `player_goals`.

If the user is returning, their saved preferences are shown as defaults. The user can press Enter to keep them or type a new answer. This works the same way in CLI (`input()`) and web (blocking on `input_q`).

generate_lesson_node — Writing the Plan

This node does the heavy lifting. It builds a large prompt that includes the Analyst's full analysis report AND the player's goals, then asks Claude to write a coaching plan in the persona of 'Coach King' — warm, specific, and grounded in real chess books.

The Retry Block

If `critique_attempts > 0`, a RETRY BLOCK is injected into the prompt *before* the coaching instructions. This tells Claude exactly what the Critic rejected and why. This is important: without this, Claude would likely write a

similar report again because it has no memory of what it did before.

```
retry_block = ""
if attempts > 0 and prev_critique.get("feedback"):
    retry_block = f""
IMPORTANT — Your previous coaching report was REJECTED by the critic.
Critic's feedback: "{prev_critique['feedback']}"
Score received: {prev_critique.get('total_score', 'N/A')}/100
Weaknesses: {prev_critique.get('weaknesses', [])}
Address every point. Do not repeat the same mistakes.
""
```

Agent 3: The Critic

File: **agents/critic.py** — function: **critic_review_node(state)**

The Critic is Claude playing a different role: quality-control judge rather than coach. It receives the coaching report, the original analysis, and the player's goals, and scores the report across four dimensions (0–25 each, 100 total).

The Four Scoring Dimensions

- **Specificity (0-25):** Does the report reference actual game patterns, or is it generic advice?
- **Goal Alignment (0-25):** Does it address the player's stated openings and focus areas?
- **Actionability (0-25):** Are the recommendations concrete with specific steps?
- **Time Pressure Handling (0-25):** Are time-pressure mistakes correctly deprioritized?

Structured JSON Output

The Critic's prompt ends with the instruction to respond **ONLY** in a specific JSON format. This is a common LLM pattern: force the model to produce machine-readable output by being extremely explicit about the schema in the prompt. The response is then parsed with `json.loads()` after extracting the JSON block with a regex.

```

# From the Critic's prompt:
# Respond in this EXACT JSON format – no other text:
# {
#   "scores": {"specificity": <0-25>, "goal_alignment": <0-25>, ...},
#   "total_score": <sum>,
#   "approved": <true if total >= 70>,
#   "feedback": "<if not approved: exactly what must change>",
#   "strengths": [...],
#   "weaknesses": [...]}
# }

# Parsing:
def _parse_critique(content: str) -> dict:
    try:
        match = re.search(r'\{.*\}', content, re.DOTALL)
        if match:
            return json.loads(match.group())
    except (json.JSONDecodeError, AttributeError):
        pass
    return {"approved": True, "total_score": 70, ...} # safe fallback

```

The should_regenerate Router

This is the conditional edge function. It is a plain Python function, not an agent. It reads `critique_result['approved']` and returns either 'regenerate' or 'approved'. LangGraph uses that string to look up the next node in the conditional edges map. There is also a hard cap: if `critique_attempts >= 3`, the Critic auto-approves to prevent an infinite loop.

```

MAX_ATTEMPTS = 3
PASS_THRESHOLD = 70

def should_regenerate(state: ChessCoachState) -> str:
    critique = state.get("critique_result", {})
    return "approved" if critique.get("approved") else "regenerate"

# In critic_review_node, before scoring:
if attempts >= MAX_ATTEMPTS:
    return {"critique_result": {"approved": True, "total_score": 65, ...}}

```

PART 4 — SUPPORTING SYSTEMS

Chess.com API (tools/chess_api.py)

Chess.com has a fully **public REST API** — no API key, no OAuth, no sign-up. You just hit <https://api.chess.com/pub/player/{username}/games/{year}/{month}> and get back a JSON array of games with PGN strings included.

The `fetch_recent_games()` function works backwards in time, month by month, until it has collected enough games. It looks back up to 6 months. This is necessary because a blitz-only player may not have played last month but did play 3 months ago.

```
# Simplified fetch loop
for months_back in range(6):
    if len(collected) >= num_games:
        break
    target = now - timedelta(days=30 * months_back)
    url = f"https://api.chess.com/pub/player/{username}/games/{target.year}/{target.month:02d}"
    resp = requests.get(url, headers={"User-Agent": "ChessCoachAgent/1.0"}, timeout=15)
    games = resp.json().get("games", [])
    if time_control != "all":
        games = [g for g in games if g.get("time_class") == time_control]
    collected.extend(games)
    time.sleep(0.4) # be polite to the API
```

PGN Parsing (tools/pgn_parser.py)

PGN (Portable Game Notation) is the standard text format for recording chess games. A typical PGN looks like:
1. e4 e5 2. Nf3 Nc6 3. Bb5 {[%clk 0:09:47]} ...

The **python-chess** library's `chess.pgn.read_game()` function parses a PGN string into a game tree object. The parser walks the move tree, extracting move SAN notation, clock times from comments, and computing whether each move was made under time pressure.

What `parse_game()` Returns

- `player_color` — 'white' or 'black' (determined by matching username to PGN headers)
- `outcome` — 'win' / 'loss' / 'draw' from the player's perspective
- `opening_eco`, `opening_name` — from PGN header tags (e.g. 'B20 Sicilian Defence')
- `move_string` — full algebraic move sequence with ■ markers for time pressure
- `time_pressure_count`, `time_pressure_move_numbers` — for the Analyst to flag

Stockfish Integration (utils/stockfish_engine.py)

Stockfish is the world's strongest open-source chess engine. It runs as a **subprocess** and communicates via the **UCI protocol** (Universal Chess Interface) — a simple text protocol where you send commands like 'position fen moves ' and receive responses like 'bestmove e2e4'. The `python-chess` library handles all of this automatically via its async engine interface.

Opening a Stockfish Process

```
# chess.engine.popen_uci() launches Stockfish as a subprocess
# and returns (transport, engine). We only need engine.
async def _open():
    _, engine = await chess.engine.popen_uci(STOCKFISH_PATH)
    return engine

# Always quit the engine in a finally block to avoid zombie processes
engine = await _open()
try:
    result = await engine.play(board, chess.engine.Limit(time=0.5))
finally:
    await engine.quit()
```

Centipawns and Quality

Stockfish evaluates positions in **centipawns** — hundredths of a pawn. A score of +100 means White is ahead by one pawn. +900 means White is winning almost certainly. A score of -10000 or +10000 means forced checkmate.

To evaluate a player's move, the engine analyses the position before and after the move and computes the **centipawn loss**: how many centipawns the player gave up compared to Stockfish's best move. The quality thresholds used in this project:

- 0–29 cp loss → good
- 30–79 cp loss → inaccuracy
- 80–199 cp loss → mistake
- 200+ cp loss → blunder

get_best_move() — Coach Moves

Used during the Play vs Coach game. The Skill Level setting (0–20) controls how strong Stockfish plays. Level 3 = easy, 8 = medium, 15 = hard, 20 = expert. Lower skill levels make Stockfish deliberately pick suboptimal moves.

analyse_move() — Real-Time Feedback

After each player move in the Play mode, `analyse_move()` runs Stockfish at full depth to evaluate the move quality. If it is a blunder or mistake (depending on coaching level setting), Claude is asked to explain the error, citing a real chess book. This runs concurrently with the coach's next move calculation using `asyncio.gather()`.

```
# Run Stockfish analysis and coach move concurrently – neither needs to wait for the other
analysis_raw, coach_mv = await asyncio.gather(
    analyse_move(chess.Board(fen_before), mv),
    get_best_move(board, difficulty),
)
```

batch_analyse_game() — Puzzle Extraction

Analysing every position in multiple games one at a time would mean opening and closing a Stockfish subprocess for each position. That is very slow (each process launch ~200ms). `batch_analyse_game()` opens **one engine instance**, runs through all positions in a loop, then quits. This is the key performance optimization for the puzzle

generator.

```
async def batch_analyse_game(positions: list, depth: int = 12) -> list:
    engine = await _open()
    results = []
    try:
        await engine.configure({"Skill Level": 20})
        for board_before, played_move in positions:
            info_before = await engine.analyse(board_before, chess.engine.Limit(depth=depth), ...)
            # ... compute cp_loss for this position
            results.append({...})
    finally:
        await engine.quit() # always clean up
    return results
```

Puzzle Generation (api/routes/practice.py)

Puzzles are extracted exclusively from the user's own real games — not from a pre-made database. This makes them personally relevant: every puzzle is a position from a game you actually played, showing you a move you missed.

The Filtering Pipeline

After `batch_analyse_game()` runs on all positions, each position is filtered through these conditions:

1. **Skip if game was already decided** — if $|\text{best_eval}| > 600$ centipawns, the position was already winning or losing by a large margin. A blunder there doesn't teach a lesson.
2. **cp_loss ≥ 100** — must be a significant miss (mistake or blunder), not just a slightly inferior move.
3. **Best move must be tactical** — the missed move must be a capture, give check, or be a promotion. This ensures the puzzle has a clear, findable solution (not a subtle positional idea).
4. **Deduplication** — at most one puzzle per opponent, and only if the move numbers are 3+ apart, to avoid showing two puzzles from the same sequence.

```
def _is_tactical(board: chess.Board, move: chess.Move) -> bool:
    return (board.is_capture(move)
            or board.gives_check(move)
            or move.promotion is not None)
```

Up to 5 puzzles are shown, sorted by `cp_loss` descending — the biggest blunders shown first. Claude Haiku (a faster, cheaper model) generates the 2-sentence explanation for each puzzle, citing one chess book.

PART 5 — THE WEB LAYER

FastAPI and WebSockets

FastAPI is a Python web framework built on Starlette and Pydantic. It handles HTTP routes (serving HTML, reports) and WebSocket connections. WebSockets are used instead of HTTP requests because the pipeline takes

60–120 seconds and streams output progressively — HTTP requests time out and can't stream.

The Three WebSocket Endpoints

- **/ws** — The main coaching session. Runs the full LangGraph pipeline.
- **/ws/game** — The Play vs Coach mode. Plays a live chess game move by move.
- **/ws/practice** — Puzzle and opening line generation.

The /ws Endpoint in Detail

This is the most complex endpoint. Here is exactly what happens from connection to completion:

1. Client connects over WebSocket. Server calls `websocket.accept()`.
2. Client sends a JSON message: `{type:'start', username:..., num_games:..., time_control:...}`
3. Server creates `event_q` and `input_q` queues, calls `bus.setup_web()` with them.
4. Server launches `run_session()` in a `ThreadPoolExecutor` background thread. This function calls `run_coaching_session_web()` which invokes the LangGraph graph.
5. The bridge loop starts: every 20ms, drain `event_q` and forward all messages to the client.
6. When Coach asks a question, `bus.ask()` blocks the background thread on `input_q.get()`.
7. Client receives a `{type:'question',...}` message, shows it to the user.
8. User types an answer and submits. Client sends `{type:'answer', value:...}` over WebSocket.
9. Bridge loop receives the answer, puts it on `input_q`. Background thread unblocks.
10. When the graph finishes, `future.done()` becomes `True`. The bridge loop drains remaining events and exits.

Play vs Coach: The Game Loop (`api/routes/play.py`)

This is separate from the LangGraph pipeline — it is a direct WebSocket handler that implements a chess game turn by turn. No LangGraph is involved here.

Why Optimistic Updates?

When the player makes a move, the server needs to: (1) validate it, (2) run Stockfish to analyse the move quality, (3) run Stockfish to pick the coach's response move. Steps 2 and 3 each take 0.5–1 second. If we waited for both before updating the board, the player would see their piece freeze for 1–2 seconds after clicking.

The fix is to send **move_ok** immediately after validation (before Stockfish). The browser updates the board instantly. Stockfish runs concurrently in the background. The mistake analysis and coach move arrive later as separate messages.

```

# Validate the move
if mv not in board.legal_moves:
    await websocket.send_json({"type": "invalid_move"})
    continue

fen_before = board.fen()
san = board.san(mv)
board.push(mv)

# 1. Confirm immediately – browser updates the board NOW
await websocket.send_json({"type": "move_ok", "san": san, "board": ..., "fen": ...})

# 2. Run both Stockfish tasks concurrently – neither waits for the other
analysis_raw, coach_mv = await asyncio.gather(
    analyse_move(chess.Board(fen_before), mv),
    get_best_move(board, difficulty),
)

# 3. Send mistake if significant (separate message, arrives after move_ok)
mistake = await build_mistake_packet(fen_before, san, mv, analysis_raw, coaching)
if mistake:
    await websocket.send_json({"type": "mistake", "mistake": mistake})

# 4. Send coach's response move
coach_san = board.san(coach_mv)
board.push(coach_mv)
await websocket.send_json({"type": "coach_move", "san": coach_san, ...})

```

Deployment: Docker + Fly.io

The Dockerfile — Line by Line

```

FROM python:3.12-slim          # minimal Debian-based Python image

# Install Stockfish from apt – this is the correct path for Linux
RUN apt-get update && apt-get install -y stockfish && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .                      # copy everything after pip install (layer cache)

# Tell the app where Stockfish lives on Linux (vs. /opt/homebrew/bin/stockfish on Mac)
ENV STOCKFISH_PATH=/usr/games/stockfish

EXPOSE 8080
CMD ["uvicorn", "api.server:app", "--host", "0.0.0.0", "--port", "8080"]

```

fly.toml

Fly.io is a platform that runs Docker containers globally. fly.toml is its configuration file. Key settings:

- **internal_port = 8080** — must match the port in the CMD.

- **force_https = true** — Fly.io handles SSL; our app only speaks plain HTTP internally.
- **auto_stop_machines = 'stop'** — the machine stops when not in use (saves money).
- **auto_start_machines = true** — it automatically restarts when a request comes in.
- **min_machines_running = 0** — allow scale-to-zero (cold start ~2-3 seconds).

The ANTHROPIC_API_KEY Secret

The app needs an Anthropic API key to call Claude. You never put secrets in the Dockerfile or fly.toml. Instead: fly secrets set ANTHROPIC_API_KEY=sk-ant-.... Fly.io injects it as an environment variable at runtime. The Anthropic SDK reads it automatically from `os.environ['ANTHROPIC_API_KEY']`.

PART 6 — HOW TO BUILD IT YOURSELF

Step 0: Project Setup

```
mkdir chess-coach && cd chess-coach
python -m venv venv && source venv/bin/activate

pip install langgraph langchain-anthropic anthropic fastapi uvicorn[standard] \
    python-chess chess requests reportlab

# Create the directory structure
mkdir agents graph tools utils api memory outputs frontend
touch agents/__init__.py graph/__init__.py tools/__init__.py \
    utils/__init__.py api/__init__.py memory/__init__.py

export ANTHROPIC_API_KEY="your-key-here"
```

Step 1: Define the State and a Minimal Graph

Start with just the state and a graph that runs one node. Verify it compiles and runs.


```
# graph/state.py
from typing import TypedDict
class State(TypedDict):
    username: str
    analysis: str

# graph/pipeline.py
from langgraph.graph import StateGraph, END, START
from graph.state import State

def dummy_node(state: State) -> dict:
    return {"analysis": f"Hello {state['username']}"}

workflow = StateGraph(State)
workflow.add_node("dummy", dummy_node)
workflow.add_edge(START, "dummy")
workflow.add_edge("dummy", END)
graph = workflow.compile()

result = graph.invoke({"username": "Brody", "analysis": ""})
print(result["analysis"]) # Hello Brody
```

Step 2: Add the EventBus

Create `utils/events.py` with the `EventBus` class as shown in Part 2. Instantiate bus at module level. Replace all `print()` calls in your nodes with `bus.debug()`. Test in CLI mode (default) — everything should still work.

Step 3: Add the Analyst Agent

Add `agents/analyst.py` with `analyze_games_node()`. Add it to the graph. Feed it a hardcoded `game_summaries` list first to test the Claude prompt before wiring up the Chess.com API.

```
# Minimal analyst – test Claude is working
from langchain_anthropic import ChatAnthropic
def analyze_games_node(state):
    llm = ChatAnthropic(model="claude-haiku-4-5-20251001", temperature=0.3)
    prompt = f"Analyze these chess games: {state['game_summaries'][:1]}"
    response = ""
    for chunk in llm.stream(prompt):
        response += chunk.content
    return {"analysis_report": response}
```

Step 4: Add the Coach + Critic Loop

Add the Coach and Critic agents. Wire up the conditional edge. Test that the Critic can reject and the Coach can revise. Print `critique_attempts` at each iteration to confirm the loop is working.

Step 5: Add the FastAPI Server

Create `api/server.py`. Add the `/ws` WebSocket endpoint with the bridge loop. Test with a simple frontend that connects and sends a start message. Add the static file serving for the frontend.

Step 6: Add Stockfish

Install Stockfish on your machine (brew install stockfish on Mac). Create `utils/stockfish_engine.py`. Test `get_best_move()` standalone before wiring it into the game loop.

```
import asyncio, chess
from utils.stockfish_engine import get_best_move

async def test():
    board = chess.Board()
    move = await get_best_move(board, "medium")
    print(board.san(move)) # e.g. "e4" or "d4"

asyncio.run(test())
```

Step 7: Deploy to Fly.io

1. Install flyctl: brew install flyctl
2. Login: fly auth login
3. In the project root: fly launch — follow the prompts, choose a region.
4. Set your secret: fly secrets set ANTHROPIC_API_KEY=sk-ant-...
5. Deploy: fly deploy
6. View logs: fly logs

Common Mistakes and How to Fix Them

- **ANTHROPIC_API_KEY not set** — langchain_anthropic will raise an AuthenticationError. Make sure the env var is exported before running.
- **Stockfish path wrong** — On Mac: `/opt/homebrew/bin/stockfish`. On Debian/Docker: `/usr/games/stockfish`. Use `STOCKFISH_PATH` env var to configure.
- **Starlette MutableHeaders has no .pop()** — Use `'del response.headers[key]'` inside a try/except KeyError instead.
- **LangGraph node returning None** — Every node must return a dict (even an empty one `{}`). Returning None will corrupt the state.
- **WebSocket disconnects on long runs** — The bridge loop's 20ms sleep and 50ms receive timeout keep the connection alive. Without these, the browser will time out.
- **Stockfish zombie processes** — Always call `engine.quit()` in a finally block. Forgetting this leaves orphaned subprocesses that consume memory.

- **asyncio.gather() with non-coroutines** — Both arguments to `asyncio.gather()` must be coroutines (async functions). If you accidentally pass a regular function, it will raise a `TypeError`.

Key Things to Understand Before Moving On

1. LangGraph's job is *control flow* — deciding which function runs next and when to loop. The AI agents are just Python functions that happen to call Claude.
2. The EventBus decouples the agents from the output medium. The same agent code works in terminal or browser.
3. Stockfish runs as a subprocess. `python-chess` handles the UCI protocol. You never write UCI commands yourself.
4. The WebSocket bridge loop is just polling: every 20ms, drain the output queue and check for new input. There is no magic.
5. The entire LangGraph graph runs synchronously in a background thread. FastAPI's async event loop never runs the graph — it only relays events.
6. Centipawn loss is the key metric for chess quality. 0-30 = good, 30-80 = inaccuracy, 80-200 = mistake, 200+ = blunder. These thresholds are industry-standard.

Glossary

LangChain — A Python library of building blocks for LLM applications. Provides ChatAnthropic, message formats, and streaming.

LangGraph — A library for defining AI workflows as directed graphs. Manages shared state and control flow between agent nodes.

StateGraph — The core LangGraph class. You add nodes and edges to it, then compile it into a runnable graph.

TypedDict — A Python type that lets you define a dictionary with named, typed fields. Used as the LangGraph state type.

Node — A Python function in a LangGraph graph. Takes state as input, returns a dict of updates.

Edge — A directed connection between two nodes. Can be unconditional (always go here) or conditional (route based on state).

EventBus — A singleton object that routes agent output to the correct destination (terminal or WebSocket).

UCI — Universal Chess Interface — the text protocol that chess GUIs use to communicate with chess engines like Stockfish.

Centipawn (cp) — One hundredth of a pawn. The unit Stockfish uses to measure position advantage.

FEN — Forsyth-Edwards Notation — a compact string that encodes a complete chess position. Used to set up puzzle boards.

PGN — Portable Game Notation — the standard format for recording a chess game, including moves and metadata.

WebSocket — A persistent two-way connection between browser and server. Unlike HTTP, either side can send messages at any time.

asyncio.gather() — Runs multiple async tasks concurrently and waits for all to complete. Used to run Stockfish analysis and move generation in parallel.

ThreadPoolExecutor — Runs blocking functions in a thread pool so they don't block the async event loop.